Project Development Phase
Model Performance Test

Date	16 February 2026
Team ID	LTVIP2026TMIDS79524
Project Name	Smart Sorting: Transfer Learning for Identifying Rotten Fruits and Vegetables
Maximum Marks	10 Marks

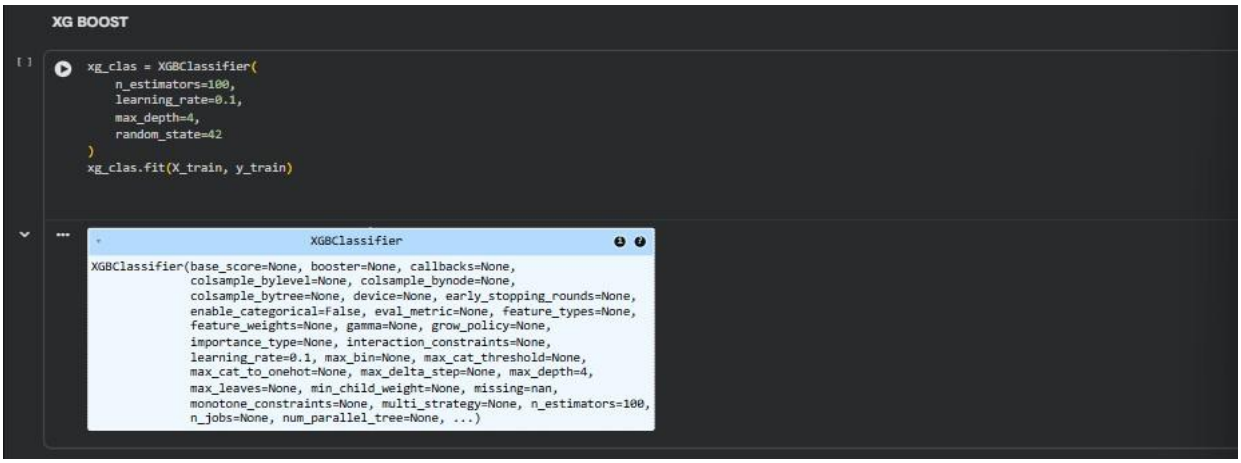
Model Building, Training, and Performance Testing

The model building phase involved developing a deep learning image classification model to predict whether a fruit is fresh or spoiled based on visual features. A convolutional neural network (CNN) architecture was implemented using Python and deep learning libraries. CNN was chosen because of its strong ability to automatically extract meaningful image features such as color variations, texture patterns, and surface characteristics.

The fruit images served as input data, while the corresponding freshness labels (Fresh / Spoiled) were used as the target classes. The model architecture was configured with appropriate layers, activation functions, and training parameters to ensure efficient learning and improved classification accuracy.

A training pipeline was created where the prepared image dataset was passed to the model using a training function. The model was trained using the .fit() method, which allowed it to learn patterns from the training images. Validation data was used to monitor performance during training. Predictions on unseen images were generated using the model's inference capability.

For evaluating performance, training accuracy and validation accuracy were monitored across epochs. Loss curves were analyzed to detect overfitting or underfitting. Additional evaluation techniques such as confusion matrix and classification metrics were used to measure how well the model distinguished between fresh and spoiled fruit categories.



Model Training

During the training phase, the fruit image dataset was divided into training and validation sets, typically using an 80:20 split ratio. The training dataset allowed the convolutional neural network

to learn distinguishing visual patterns associated with fresh and spoiled fruits, such as color variations, texture irregularities, and surface characteristics.

The training process involved feeding batches of preprocessed images into the network across multiple epochs. During each epoch, the model adjusted its internal weights using backpropagation and optimization techniques to minimize classification error. Training parameters such as learning rate, batch size, and number of epochs were tuned to improve accuracy while preventing overfitting. Validation data was continuously monitored to ensure the model generalized well to unseen images.

Performance Testing

After training, the model was evaluated using the validation/testing dataset to measure its ability to classify new fruit images accurately. Several evaluation indicators were used, including:

- Accuracy — measures overall correctness of freshness predictions
- Precision — evaluates how reliably the model predicts a fruit as fresh or spoiled
- Recall — measures the model's ability to correctly detect actual freshness categories
- F1-Score — balances precision and recall for overall reliability
- Confusion Matrix — analyzes correct and incorrect classifications across categories

These evaluation methods ensured the model performed consistently and reliably in real-world scenarios.

```
[ ]
from sklearn import metrics
import pandas as pd

results = []

models = {
    "KNN": p1,
    "Decision Tree": p2,
    "Random Forest": p3,
    "XGBoost": p4
}

for name, pred in models.items():
    acc  = metrics.accuracy_score(y_test, pred)
    prec = metrics.precision_score(y_test, pred)
    rec  = metrics.recall_score(y_test, pred)
    f1   = metrics.f1_score(y_test, pred)

    results.append([name, acc, prec, rec, f1])

results_df = pd.DataFrame(results,
                          columns=["Model","Accuracy","Precision","Recall","F1-Score"])

print(results_df)
```

```
            Model  Accuracy  Precision    Recall  F1-Score
0             KNN  0.896552        0.5  1.000000  0.666667
1   Decision Tree  0.965517        1.0  0.666667  0.800000
2   Random Forest  0.965517        1.0  0.666667  0.800000
3         XGBoost  0.965517        1.0  0.666667  0.800000
```

Comparative analysis of all algorithms was performed. Among them, XGBoost achieved the highest accuracy and better overall performance metrics. It also demonstrated strong ability in handling nonlinear relationships and minimizing prediction errors.

Based on these results, XGBoost was selected as the final model and deployed into the web application for real-time flood risk prediction.